Báo cáo Lab_w05

Họ và tên: Phạm Đình Vũ
Mã sinh viên: 202419021
Link GitHub Repository: `https://github.com/dinhvu07112k6-cmyk/lab_w05.git`

October 11, 2026

PHẦN 1: PHÂN TÍCH YÊU CẦU NGHIỆP VỤ

a Mục tiêu hệ thống

Xây dựng hệ thống quản lý danh mục thiết bị trong các phòng máy của trường đại học và hỗ trợ dự toán chi phí bảo trì hằng năm.

b Các nhóm thiết bị

Hệ thống quản lý ba nhóm thiết bị:

- **Máy tính (Computer)**
- **Máy in (Printer)**
- **Máy chiếu (Projector)**

c Thông tin và quy tắc chung

Mỗi thiết bị có các thông tin cơ bản:

- Mã thiết bị và tên thiết bị.
- Năm đưa vào sử dụng và giá mua.
- Trạng thái: *Active* (đang hoạt động), *UnderMaintenance* (đang bảo trì) hoặc *Retired* (đã ngừng sử dụng).

Khi tạo và cập nhật thiết bị, cần đảm bảo:

- Mã thiết bị không được để trống và chỉ được gán lúc tạo thiết bị.
- Giá mua phải lớn hơn 0.
- Năm đưa vào sử dụng không được lớn hơn năm hiện tại.

d Cách tính chi phí bảo trì

Chi phí được tính riêng theo từng loại thiết bị, dựa trên giá mua và tình trạng sử dụng:

- **Máy tính:** 5% giá mua; cộng thêm 2% nếu có GPU rời và 1% nếu đã sử dụng trên 5 năm.
- **Máy in:** 4% giá mua; cộng 500.000 đồng nếu đã in trên 100.000 trang và 300.000 đồng nếu là máy in màu.
- **Máy chiếu:** 3% giá mua; cộng 1.500.000 đồng nếu bóng đèn đã hoạt động trên 3.000 giờ.

Hệ thống cần tự tính tổng chi phí bảo trì của phòng từ danh sách thiết bị. Mỗi loại thiết bị tự áp dụng công thức tương ứng, không cần phân nhánh theo loại bằng `if`/`switch`.

e Kết nối mạng

Khả năng kết nối mạng tùy theo loại thiết bị:

- Máy tính luôn có kết nối mạng.
- Một số máy in hỗ trợ kết nối mạng.
- Máy chiếu không có kết nối mạng.

Với thiết bị có hỗ trợ mạng, cần tuân thủ các quy tắc sau:

- Địa chỉ IP không được để trống khi kết nối.
- Không cho phép kết nối lại nếu thiết bị đang được kết nối.
- Khi ngắt kết nối, phải xóa địa chỉ IP đã lưu.

f Quản lý phòng thực hành (LabRoom)

Mỗi phòng thực hành quản lý một danh sách thiết bị. Khi thêm thiết bị vào phòng:

- Không chấp nhận thiết bị `null`.
- Không cho phép hai thiết bị có cùng mã trong một phòng.

Phòng thực hành cũng cần hỗ trợ lọc danh sách thiết bị cần bảo trì. Một thiết bị được đưa vào danh sách này nếu đang ở trạng thái *UnderMaintenance* hoặc đã được sử dụng trên 5 năm.

PHẦN 2: DANH SÁCH LỚP VÀ SƠ ĐỒ QUAN HỆ

a Danh sách lớp và thành phần chính

- **Device:** Lớp cơ sở trừu tượng, lưu mã, tên, năm sử dụng, giá mua và trạng thái. Lớp khai báo phương thức tính chi phí bảo trì hằng năm để các lớp thiết bị cụ thể tự cài đặt.
- **Computer:** Lưu cấu hình RAM, CPU và thông tin có GPU rời hay không; có chức năng kết nối và ngắt kết nối mạng.
- **Printer:** Lưu loại máy in, số trang đã in và thông tin máy in màu hay không.
- **NetworkPrinter:** Máy in có hỗ trợ mạng, quản lý địa chỉ IP và trạng thái kết nối.
- **Projector:** Lưu độ sáng và số giờ bóng đèn đã hoạt động.
- **LabRoom:** Lưu mã phòng, tên phòng, sức chứa và danh sách thiết bị; hỗ trợ thêm, xóa, tìm thiết bị, tính tổng chi phí bảo trì và lọc thiết bị cần bảo trì.

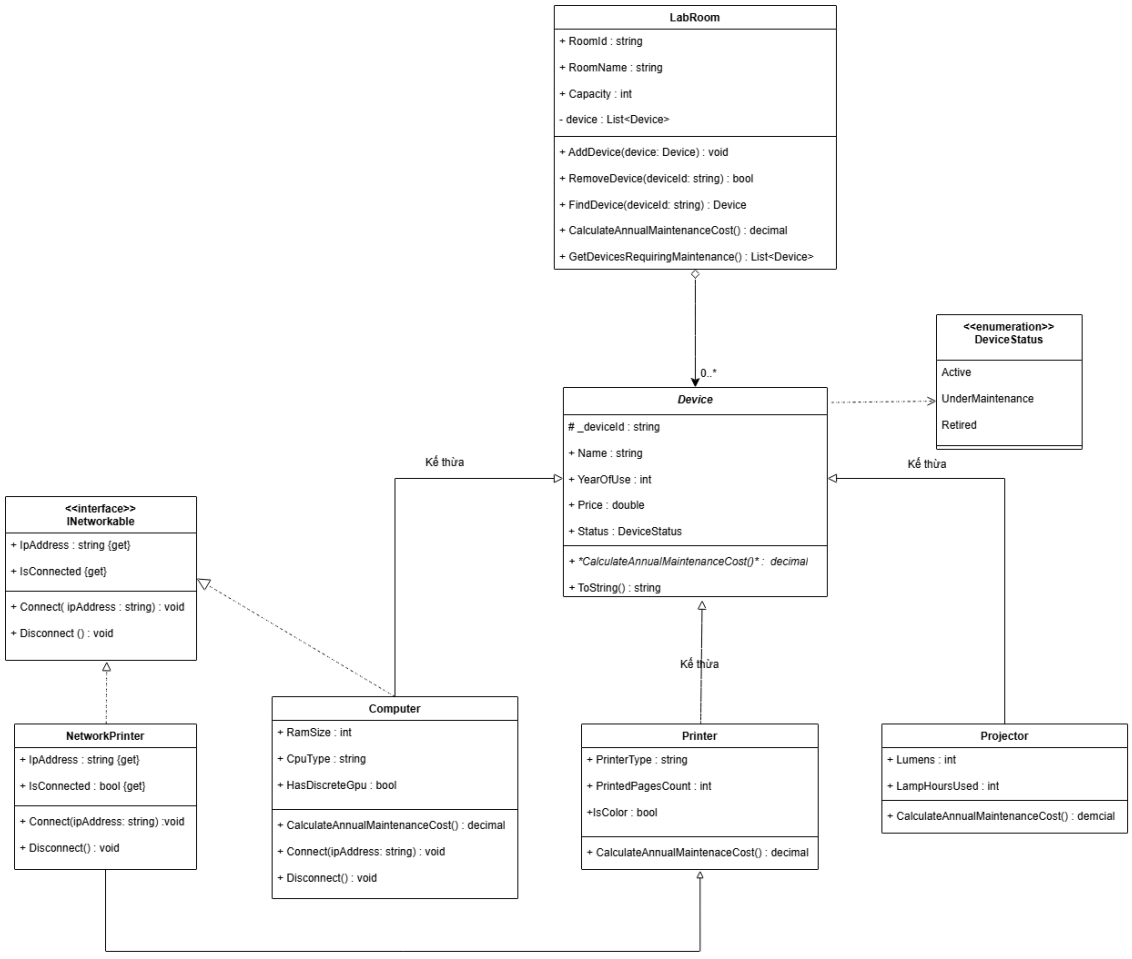
- **INetworkable:** Giao diện quy định các thông tin và thao tác kết nối mạng gồm địa chỉ IP, trạng thái kết nối, kết nối và ngắt kết nối.
- **DeviceStatus:** Kiểu liệt kê các trạng thái *Active*, *UnderMaintenance* và *Retired*.

b Quan hệ giữa các lớp

- `Computer`, `Printer` và `Projector` kế thừa các thông tin, hành vi chung từ `Device`; mỗi lớp có cách tính chi phí bảo trì riêng.
- `NetworkPrinter` kế thừa từ `Printer` và triển khai giao diện `INetworkable`. `Computer` cũng triển khai giao diện này.
- `LabRoom` quản lý danh sách gồm không hoặc nhiều đối tượng `Device`.
- Thuộc tính trạng thái của `Device` sử dụng kiểu `DeviceStatus`.

c Sơ đồ lớp

Sơ đồ dưới đây thể hiện các lớp, thuộc tính, phương thức và quan hệ kế thừa, kết nối mạng giữa chúng.



PHẦN 3: TRẢ LỜI CÁC CÂU HỎI THIẾT KẾ

a Vì sao `Device` là lớp trừu tượng?

- `Device` mô tả thiết bị nói chung, không phải một thiết bị cụ thể để tạo đối tượng trực tiếp.
- Mỗi loại thiết bị có công thức bảo trì riêng, vì vậy lớp con cần tự cài đặt phương thức tính chi phí bảo trì.
- Lớp trừu tượng giúp quản lý các loại thiết bị chung trong một danh sách và hỗ trợ tính đa hình.

b Vì sao `INetworkable` là giao diện?

- Kết nối mạng là một khả năng, không phải quan hệ kế thừa.
- `Computer` và `NetworkPrinter` có thể cùng triển khai giao diện này, dù chúng có quan hệ kế thừa khác nhau.
- Chỉ các lớp có khả năng kết nối mạng mới cần cài đặt `Connect()`, quản lý địa chỉ IP và hỗ trợ ngắt kết nối.

c Tính đa hình được thể hiện ở đâu?

- **Qua lớp cơ sở:** `LabRoom.CalculateAnnualMaintenanceCost()` gọi phương thức tính phí trên từng thiết bị. Chương trình tự chọn cách tính tương ứng với lớp con thực tế.
- **Qua giao diện:** Có thể quản lý các đối tượng mạng trong một danh sách `INetworkable` và gọi `Connect()` mà không cần biết chính xác loại thiết bị.

d Quan hệ giữa `LabRoom` và `Device` là gì?

- Đây là quan hệ **kết tập (Aggregation)**, được thể hiện bằng hình thoi rỗng ở phía `LabRoom` trong sơ đồ.
- Thiết bị có thể tồn tại độc lập trước khi được thêm vào phòng.
- Khi phòng bị xóa, thiết bị vẫn tồn tại và có thể được chuyển sang phòng khác.

PHẦN 4: KẾT QUẢ CHẠY CHƯƠNG TRÌNH

a Thêm thiết bị và kiểm tra mã trùng

- Đã thêm 5 thiết bị vào 2 phòng: 2 máy tính, 2 máy in và 1 máy chiếu.
- Thêm trùng mã `PC01` vào `LAB1` bị từ chối đúng quy tắc.

```
--- 1. Them thiet bi vao cac phong ---
Da them 3 thiet bi vao Phong May Tinh 1
Da them 2 thiet bi vao Phong Da Phuong Tien

--- 2. Thu them thiet bi trung ma PC01 vao LAB1 ---
Bat loi thanh cong: Lỗi: Thiết bị có mã 'PC01' đã tồn tại trong phòng LAB1.
```

b Danh sách, chi phí và thiết bị cần bảo trì

Đã in danh sách thiết bị theo từng phòng. Tổng chi phí dự kiến là 3.370.000 VNĐ cho `LAB1` và 2.840.000 VNĐ cho `LAB2`. Thiết bị cần bảo trì: `PJ01` (đang bảo trì) và `PC02` (đã sử dụng trên 5 năm).

```
--- 3. Danh sach thiet bi tung phong ---
[LAB1] Phong May Tinh 1:
  + [PC01] Dell OptiPlex | Năm SD: 2024 | Giá: 15.000.000 VNĐ | Trạng thái: Active | Máy tính: Core i5, 16GB RAM, GPU rời: False | Mạng: Chưa kết nối
  + [PR01] Canon LBP 2900 | Năm SD: 2022 | Giá: 3.500.000 VNĐ | Trạng thái: Active | Máy in: Laser, In đen trắng, Đã in: 120.000 trang | Mạng: Không hỗ trợ
  + [PJ01] Sony VPL | Năm SD: 2021 | Giá: 16.000.000 VNĐ | Trạng thái: UnderMaintenance | Máy chiếu: 3500 Lumens, Giờ bóng đèn: 3.200h
[LAB2] Phong Da Phuong Tien:
  + [PC02] HP Workstation | Năm SD: 2018 | Giá: 28.000.000 VNĐ | Trạng thái: Active | Máy tính: Core i7, 32GB RAM, GPU rời: True | Mạng: Chưa kết nối
  + [PR02] Epson L8050 | Năm SD: 2023 | Giá: 7.500.000 VNĐ | Trạng thái: Active | Máy in: Phun, In màu, Đã in: 20.000 trang | Mạng: Không hỗ trợ | Mạng: Có hỗ trợ (Chưa kết nối)

--- 4. Tong chi phi bao tri tung phong ---
Chi phi phong 1 (Phong May Tinh 1): 3.370.000 VND
Chi phi phong 2 (Phong Da Phuong Tien): 2.840.000 VND

--- 5. Cac thiet bi can bao tri (Trang thai dang bao tri hoac > 5 nam) ---
Phong Phong May Tinh 1:
  * [PJ01] Sony VPL - Nam SD: 2021 - Trạng thái: UnderMaintenance
Phong Phong Da Phuong Tien:
  * [PC02] HP Workstation - Nam SD: 2018 - Trạng thái: Active
```

c Kết nối mạng qua giao diện

Ba thiết bị mạng được kết nối và duyệt qua `INetworkable` với các IP `192.168.1.10`, `192.168.1.11` và `192.168.1.20`. Ngắt kết nối `PR02` sẽ đặt `IsConnected = False` và xóa IP.

```
--- 6 & 7. Ket noi va duyet danh sach thiet bi qua Interface INetworkable ---
- Da ket noi: True | Dia chi IP: 192.168.1.10
- Da ket noi: True | Dia chi IP: 192.168.1.11
- Da ket noi: True | Dia chi IP: 192.168.1.20

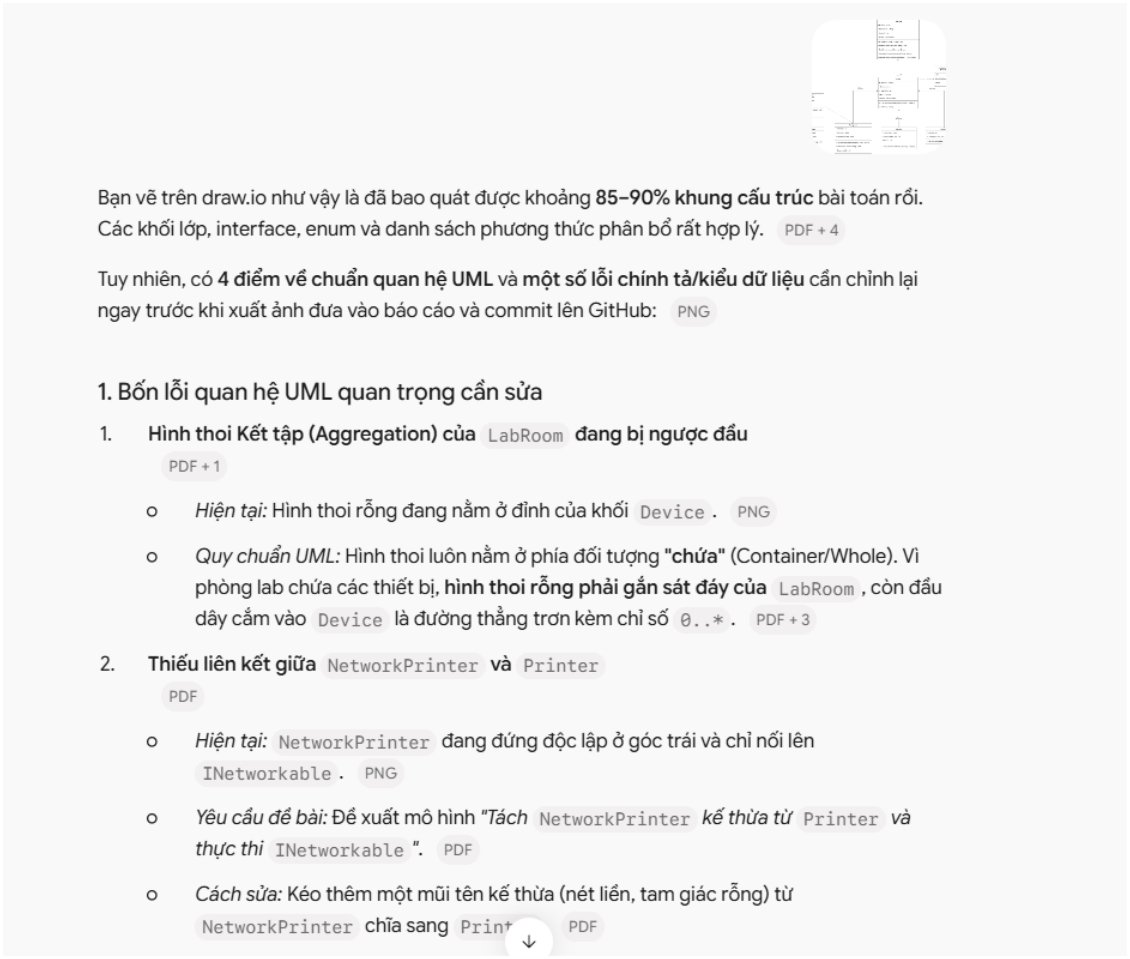
Thu ngat ket noi may in PR02:
- PR02 sau khi ngat: IsConnected = False, IP = ''
```

PHẦN 5: SỬ DỤNG AI & CHẤT LƯỢNG

a Prompt 1: Nhờ AI phản biện sơ đồ UML

Em chủ động vẽ sơ đồ lớp trên draw.io trước khi lập trình, rồi nhờ AI kiểm tra ký hiệu UML. AI đánh giá sơ đồ đã bao quát khoảng 85–90% yêu cầu và chỉ ra các điểm cần sửa: đặt hình thoi kết tập ở phía `LabRoom`, nối `NetworkPrinter` kế thừa `Printer`, hướng mũi tên thực thi vào `INetworkable`, và dùng nét liền cho quan hệ kế thừa `Printer`–`Device`. AI cũng lưu ý một số lỗi chính tả và kiểu dữ liệu `decimal`.

Em tự chỉnh sửa sơ đồ trên draw.io thay vì nhờ AI vẽ lại. Qua đó, em hiểu rõ hơn sự khác nhau giữa kế thừa, thực thi giao diện và kết tập.



Bạn vẽ trên draw.io như vậy là đã bao quát được khoảng **85–90% khung cấu trúc** bài toán rồi. Các khối lớp, interface, enum và danh sách phương thức phân bổ rất hợp lý. PDF + 4

Tuy nhiên, có **4 điểm về chuẩn quan hệ UML** và **một số lỗi chính tả/kiểu dữ liệu** cần chỉnh lại ngay trước khi xuất ảnh đưa vào báo cáo và commit lên GitHub: PNG

1. Bốn lỗi quan hệ UML quan trọng cần sửa

1. **Hình thoi Kết tập (Aggregation) của `LabRoom` đang bị ngược đầu** PDF + 1
   - *Hiện tại:* Hình thoi rỗng đang nằm ở đỉnh của khối `Device`. PNG
   - *Quy chuẩn UML:* Hình thoi luôn nằm ở phía đối tượng **"chứa"** (Container/Whole). Vì phòng lab chứa các thiết bị, **hình thoi rỗng phải gắn sát đáy của** `LabRoom`, còn đầu dây cắm vào `Device` là đường thẳng trơn kèm chỉ số `0..*`. PDF + 3
2. **Thiếu liên kết giữa `NetworkPrinter` và `Printer`** PDF
   - *Hiện tại:* `NetworkPrinter` đang đứng độc lập ở góc trái và chỉ nối lên `INetworkable`. PNG
   - *Yêu cầu đề bài:* Đề xuất mô hình *"Tách* `NetworkPrinter` *kế thừa từ* `Printer` *và thực thi* `INetworkable` *"*. PDF
   - *Cách sửa:* Kéo thêm một mũi tên kế thừa (nét liền, tam giác rỗng) từ `NetworkPrinter` chĩa sang `Print` PDF

Nhận xét của AI về sơ đồ lớp UML

b Prompt 2: Chỉ gán mã thiết bị lúc khởi tạo

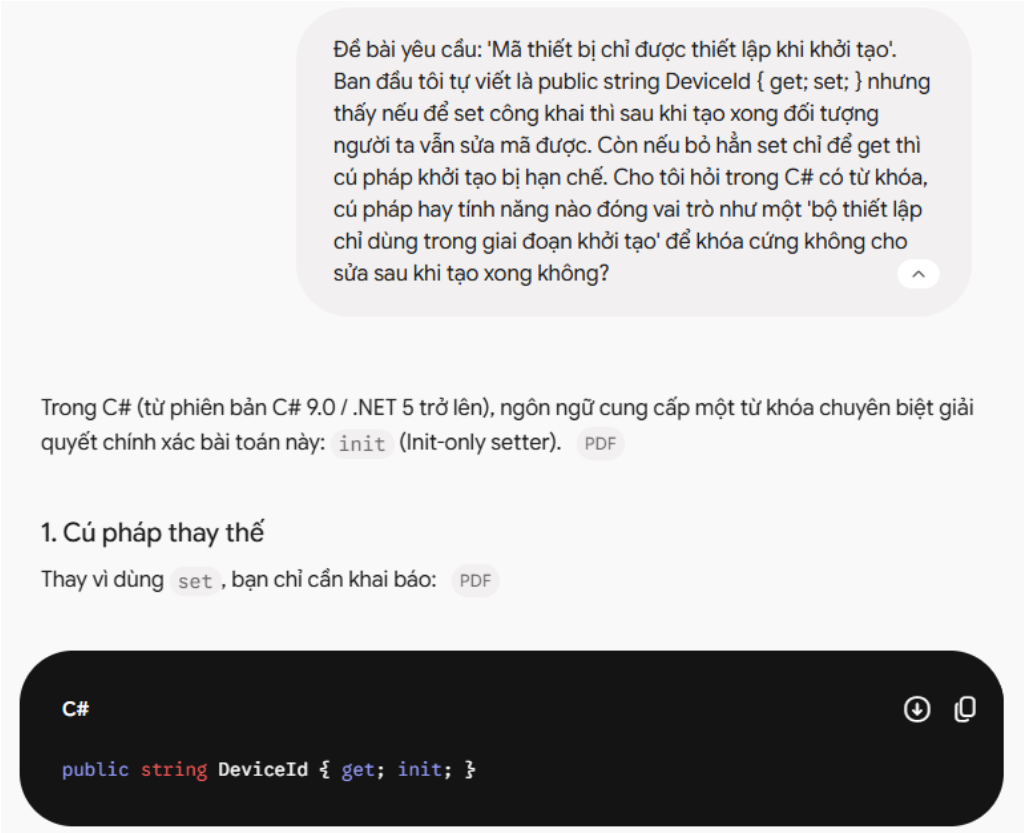
Ban đầu em dùng `DeviceId { get; set; }`, nên mã vẫn có thể bị sửa sau khi tạo. Em hỏi AI cách chỉ gán mã lúc khởi tạo; AI gợi ý dùng `init`. Sau khi đối chiếu với yêu cầu, em thấy cách này phù hợp và chọn `DeviceId { get; init; }` để mã không bị thay đổi về sau.

Em nhận thấy `init` là tính năng của C# hiện đại. Dù `private set` thường gặp trong OOP cơ bản, em chọn `init` vì đáp ứng yêu cầu và vẫn cho phép gán mã khi khởi tạo đối tượng.

AI giải thích rằng `init` cho phép gán giá trị trong constructor hoặc lúc khởi tạo đối tượng; sau đó, nếu cố sửa thuộc tính, chương trình sẽ báo lỗi khi biên dịch. So với `private set`, `init` vẫn cho phép dùng object initializer.

```
public string DeviceId { get; set; }
```

Khai báo ban đầu của thuộc tính mã thiết bị



Câu hỏi và gợi ý nhận được từ AI

2. Cách thức hoạt động

Từ khóa `init` biến thuộc tính thành dạng **chỉ được thiết lập trong pha khởi tạo đối tượng**, sau đó sẽ bị khóa cứng thành bất biến (immutable): PDF

- **Cho phép gán:** Trong constructor hoặc thông qua cú pháp khởi tạo đối tượng (Object Initializer): PDF

```C#
// Cách 1: Gán qua Constructor
var pc1 = new Computer("PC001", ...);

// Cách 2: Gán qua Object Initializer khi khởi tạo
var pc2 = new Computer(...)
{
    DeviceId = "PC002"
};
```

- **Bị chặn tuyệt đối sau khi khởi tạo:**

```C#
pc1.DeviceId = "PC999";
// Lỗi biên dịch ngay tại compile-time:
// Error CS8852: Init-only property 'Device.DeviceId' can only be assigned in an
```

AI giải thích cách gán và giới hạn của ***init***

3. So sánh 3 cách tiếp cận trong C#

Cách viết	Gán qua Constructor	Gán qua Object Initializer ({ })	Sửa đổi sau kh
`public get; set;`	Có	Có	**Có (Bị lộ, dễ sử**
`public get; private set;`	Có	**Không**	Không (ở bên n
`public get; init;`	**Có**	**Có**	**Bị cấm hoàn to (Compile error**

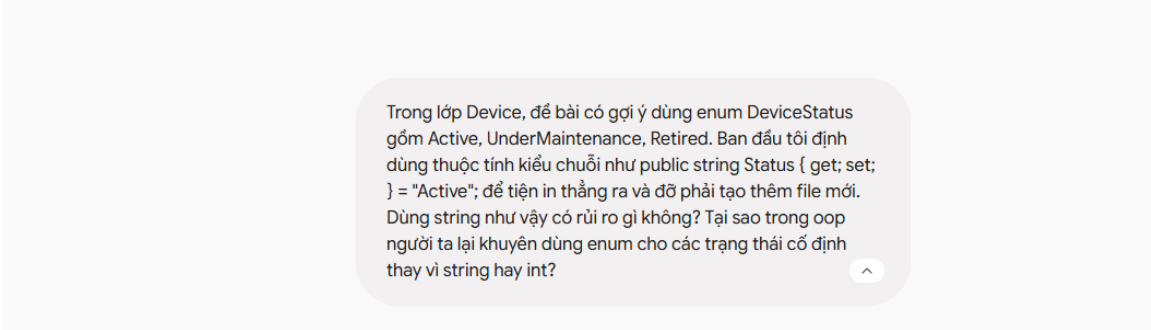
Sử dụng `{ get; init; }` vừa bảo đảm tính toàn vẹn dữ liệu ở cấp độ trình biên dịch, vừa giữ được tính linh hoạt khi viết mã. PDF

AI so sánh ***init*** *với các cách khai báo khác*

c Prompt 3: Chọn kiểu `enum` cho trạng thái

Ban đầu em định dùng `string` để lưu trạng thái cho đơn giản. AI phân tích rằng chuỗi tự do dễ bị gõ sai hoặc không thống nhất chữ hoa, chữ thường; các lỗi này có thể làm sai logic mà trình biên dịch không phát hiện. AI cũng khuyên không dùng số nguyên vì khó đọc và dễ gán nhầm giá trị.

Sau khi cân nhắc, em chọn `enum DeviceStatus` gồm `Active`, `UnderMaintenance` và `Retired`. Cách này giúp mã rõ ràng, có gợi ý khi viết code; khi in ra, `ToString()` chuyển trạng thái thành tên tương ứng. Em quyết định tách kiểu này vào file `DeviceStatus.cs` theo yêu cầu đề bài.



Câu hỏi về việc dùng `string` hay `enum`



AI so sánh các cách biểu diễn trạng thái

2. Vì sao cũng không nên dùng số nguyên (`int`)?

Nếu quy ước `0 = Active`, `1 = UnderMaintenance`, `2 = Retired`:

- **Mất tính tường minh (Poor Readability):** Đọc code `if (d.Status == 1)` sẽ không ai hiểu số `1` nghĩa là gì nếu không đọc tài liệu hoặc comment giải thích.
- **Nguy cơ gán số ngoài phạm vi:** Người dùng vẫn có thể gán `Status = 99;` mà trình biên dịch không cảnh báo.

3. Lợi ích khi sử dụng `enum DeviceStatus`

```C#
public enum DeviceStatus
{
    Active,
    UnderMaintenance,
    Retired
}
```

- **Kiểm tra chặt chẽ ngay khi biên dịch (Compile-time Verification):**
  - Bạn chỉ được phép chọn 1 trong 3 giá trị đã định nghĩa sẵn. Gõ sai một ký tự (ví dụ `DeviceStatus.Actve`), Visual Studio sẽ gạch chân đỏ báo lỗi cú pháp ngay lập tức. PDF
- **Hỗ trợ gợi ý mã nguồn (IntelliSense):**
  - Khi gõ `Status = DeviceStatus.`, II tự động xổ ra danh sách để bạn chọn bằng phím mũi tên hoặc phím Tab, giảm thiểu thời gian gõ và tránh sai sót.

AI giải thích lợi ích của `enum DeviceStatus`

PDF

- **Hỗ trợ gợi ý mã nguồn (IntelliSense):**
  - Khi gõ `Status = DeviceStatus.`, IDE sẽ tự động xổ ra danh sách để bạn chọn bằng phím mũi tên hoặc phím Tab, giảm thiểu thời gian gõ và tránh sai sót.
- **Hiển thị trực quan và tự động chuyển đổi thành chuỗi:**
  - Lầm tưởng lớn nhất là nghĩ `enum` khó in ra màn hình. Trong C#, `enum` tự động hiển thị tên định danh của nó khi in ấn hoặc dùng nội suy chuỗi:

```C#
DeviceStatus st = DeviceStatus.Active;
Console.WriteLine($"Trạng thái: {st}"); // Tự động in ra: "Trạng thái: Acti
```

Hàm `ToString()` của C# tự động phân giải tên của enum mà bạn không cần phải viết thêm bất kỳ dòng code chuyển đổi nào. PDF

Gợi ý mã và cách hiển thị trạng thái